

# Introduction to OS161

Virtual Memory

Threads

Load Exe

# Outline

- Mips registers
- OS161 threads and syncro. Primitives
- Syscalls & traps
- Address spaces and address translation
- ELF files and exec load

# Os161

- teaching operating system (written in C) a simplified unix BSD-like OS
- runs on a simulator (MIPS VM).
- two supported branches
  - 1.x branch, uniprocessor kernel
  - 2.x branch, fully released in 2015, multiprocessor support and other
- includes both a kernel of conventional ("macrokernel") design and a simple userland, including a variety of test programs.

Development tools:

- make
- configure
- Gdb
- ...

User programs  
(ELF exe)

OS161

SYS161  
(MIPS VM)



# OS161 support

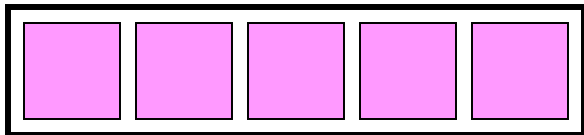
- The base OS161 system provides low-level trap/interrupt, device drivers, in-kernel threads, a baseline scheduler, a minimal virtual memory system, a simple file system
- Other things (not included) are to be implemented:
  - Locks.
  - System calls.
  - Virtual memory. The "dumbvm" shipped with OS161 is good enough for bootstrapping and doing the early assignments. It never reuses memory and cannot support large processes or malloc.
  - File system.
- Many other things can be added to OS161

# What is a thread (process) ?

- Represents the control state of an executing program
- Has an associated *context* (state)
  - Processor's CPU state: program counter (PC), stack pointer, other registers, execution mode (privileged/non-privileged)
  - Stack, located in the address space of the process
- Memory
  - Program code (out of context)
  - Program data (out of context)
  - Program stack containing procedure activation records (within context)

# Thread context

memory



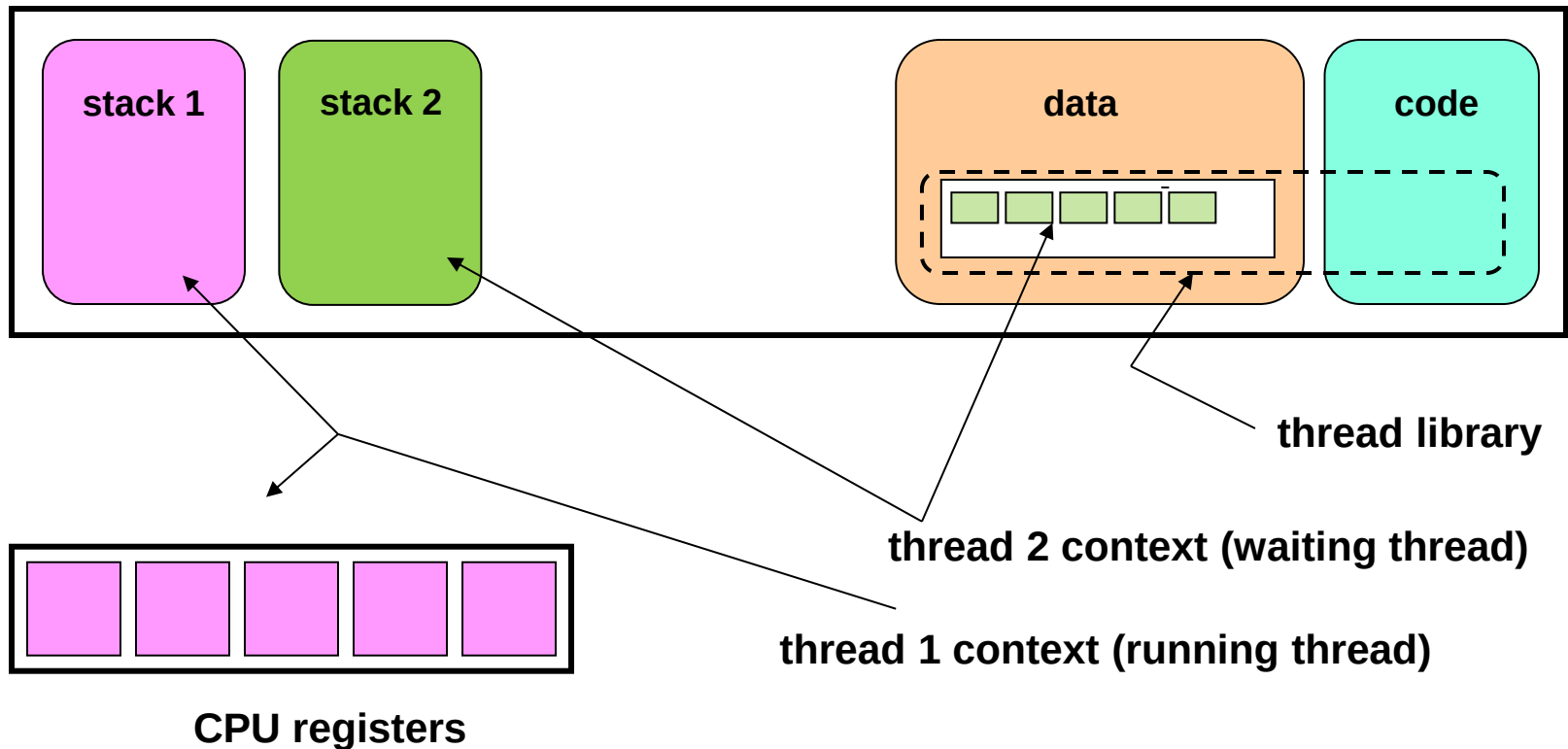
CPU registers

# Implementing threads

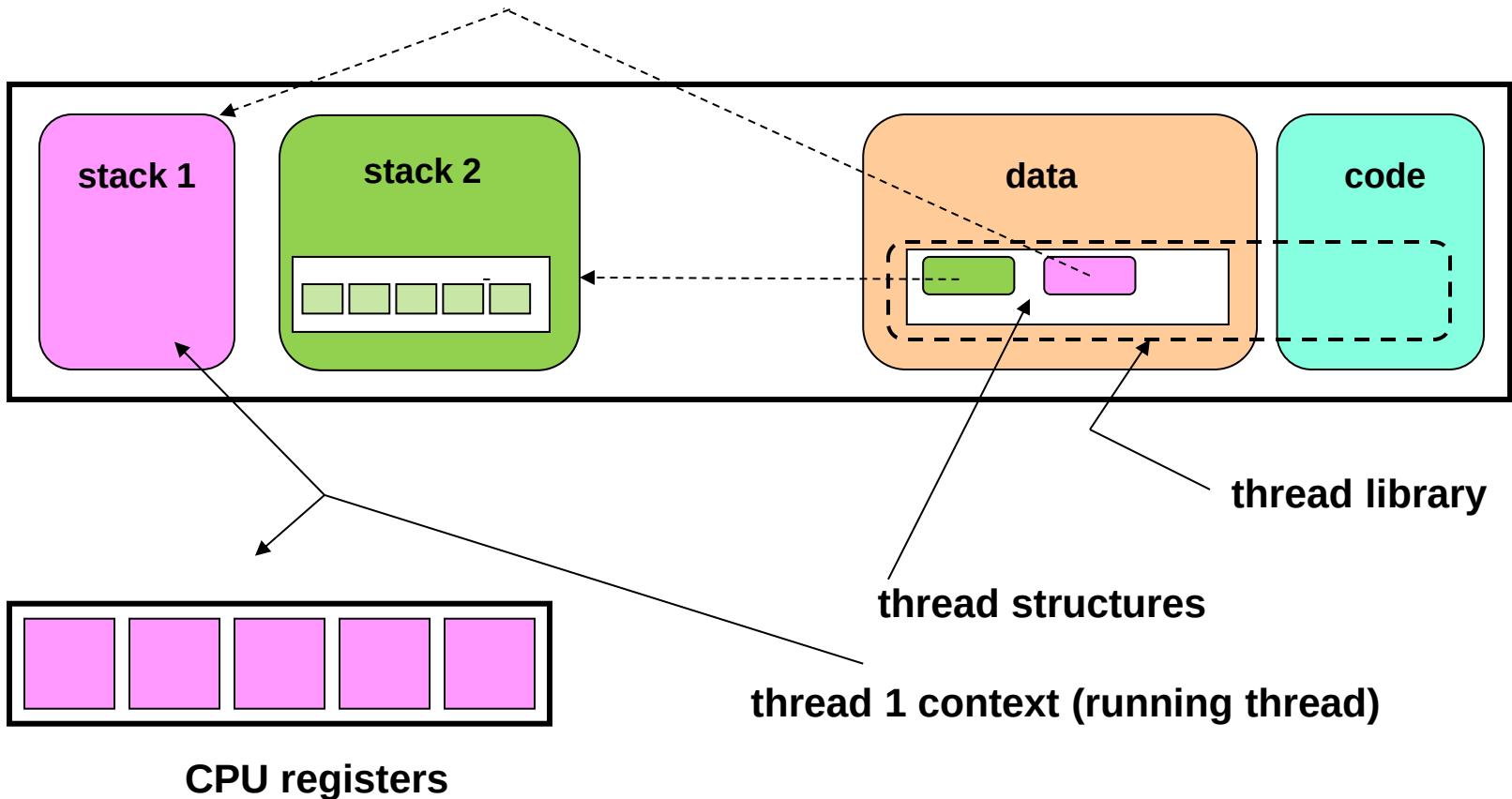
- Threads are implemented by a thread library
- Thread library stores threads' contexts (or pointers to threads' contexts) when not running
- Data structure used by thread library to store a thread context
  - Thread control block
- In OS161 thread control block is a ***thread*** structure



# Thread library and two threads (generic)



# Thread library and two threads (OS161)

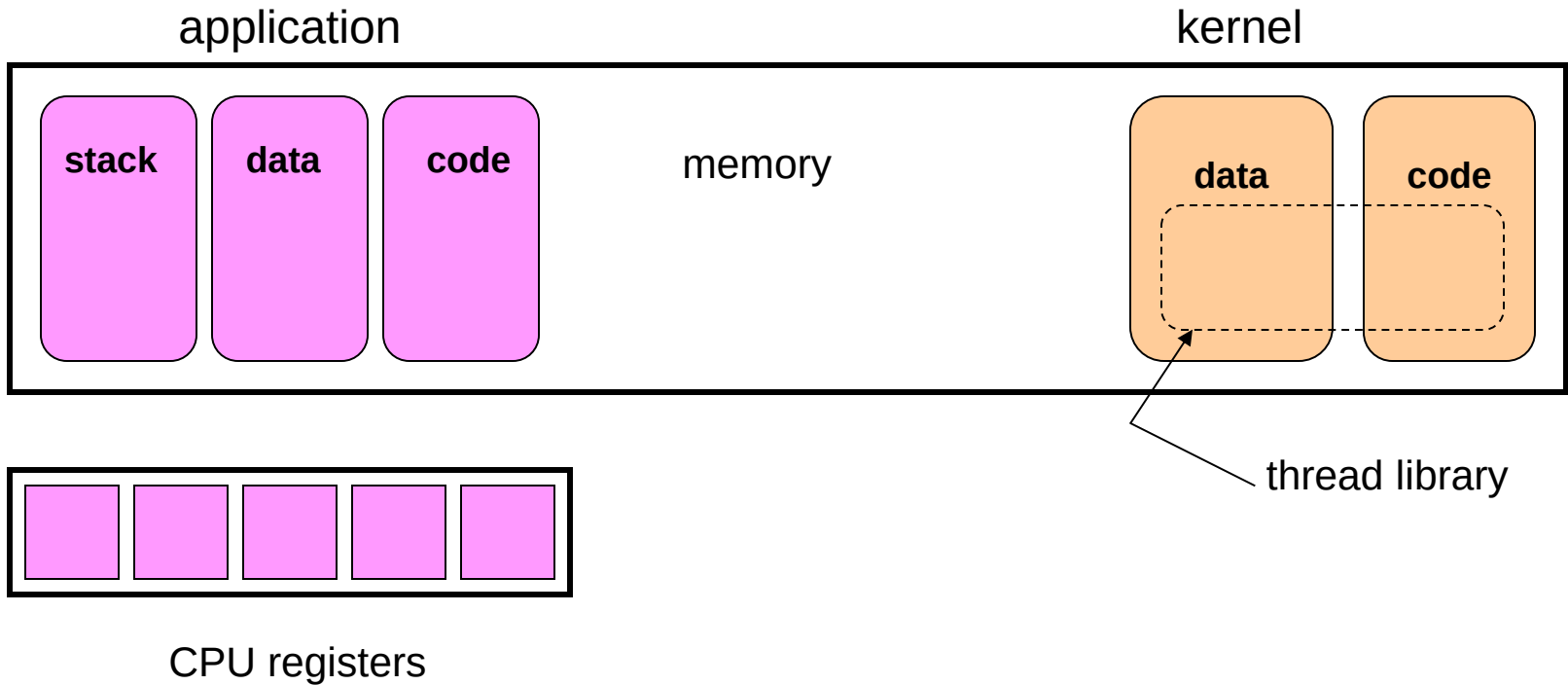


# The OS161 thread Structure

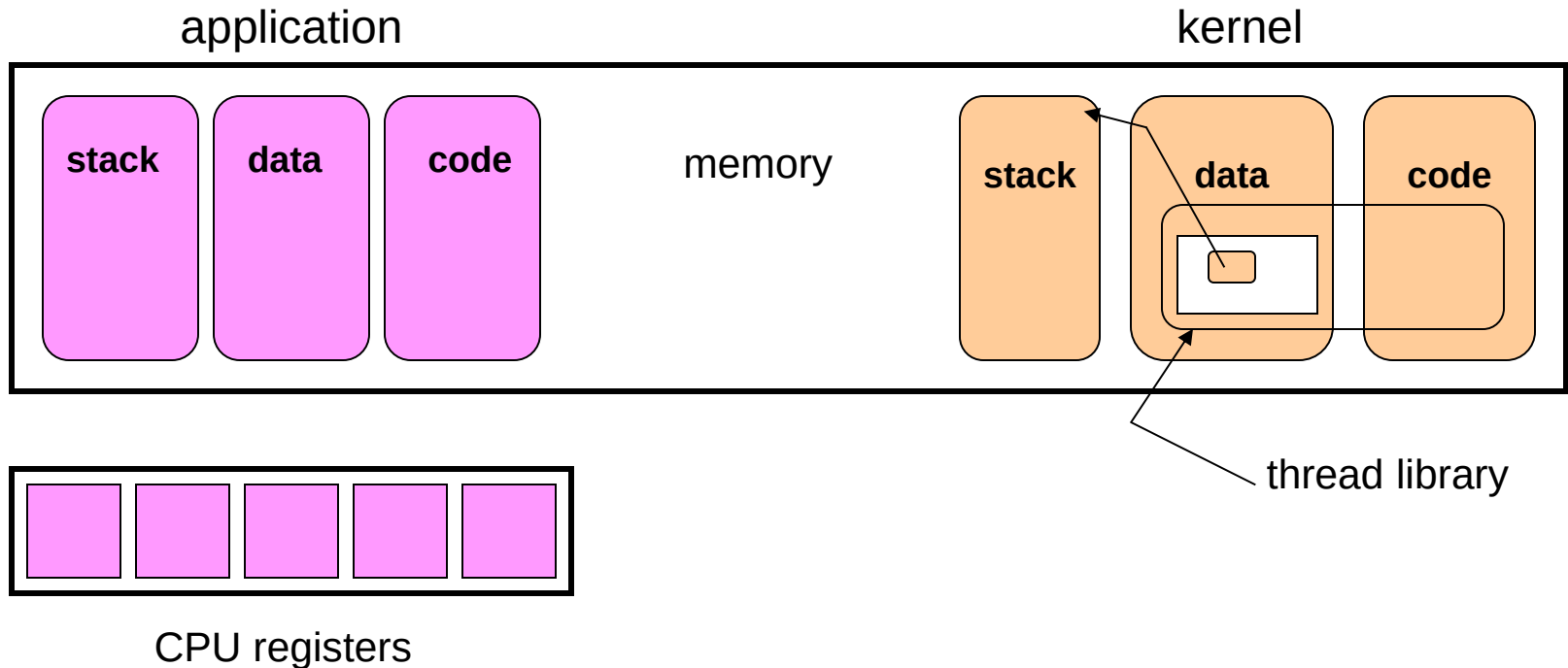
```
/* see kern/include/thread.h */
```

```
struct thread {
    char *t_name;                /* Name of this thread */
    const char *t_wchan_name; /* Name of wait channel, if sleeping */
    threadstate_t t_state;      /* State this thread is in */
    /* Thread subsystem internal fields. */
    struct thread_machdep t_machdep;
    struct threadlistnode t_listnode;
    void *t_stack;               /* Kernel-level stack */
    struct switchframe *t_context; /* Saved register context (on
    stack) */
    struct cpu *t_cpu;           /* CPU thread runs on */
    struct proc *t_proc;        /* Process thread belongs to */
    ...
};
```

# Application and kernel

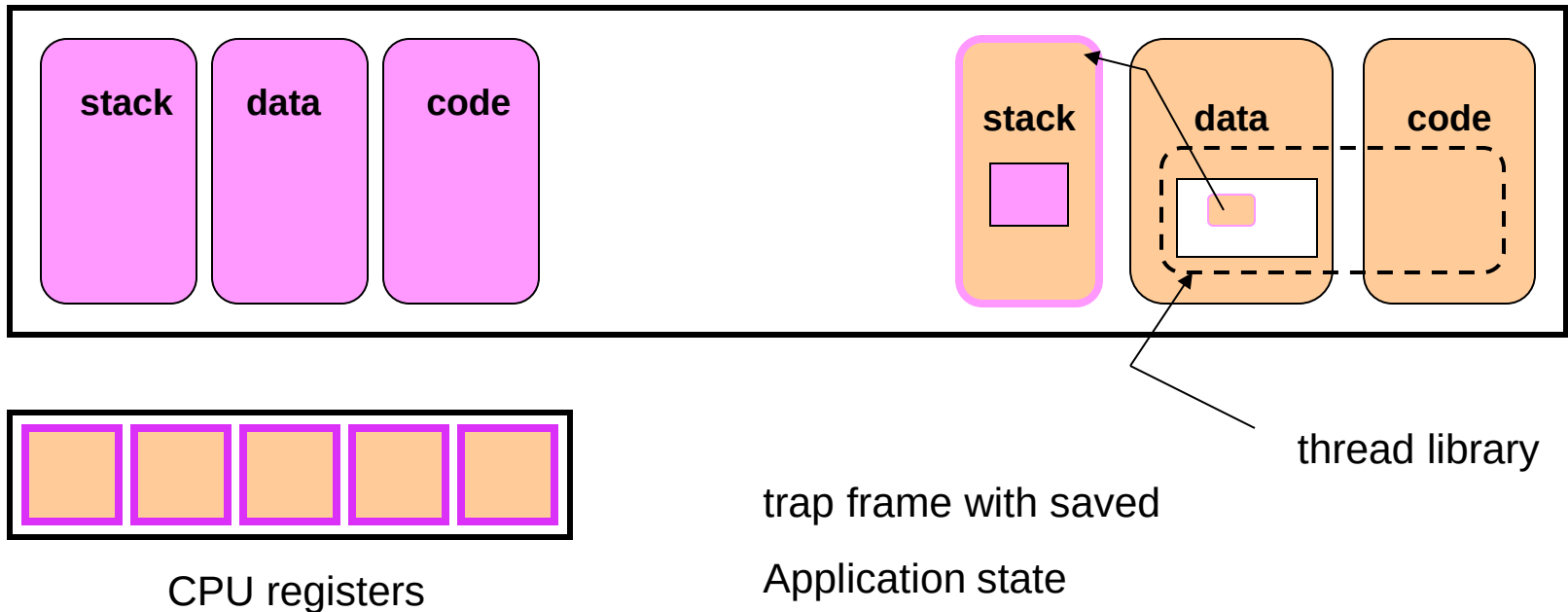


# OS161 User and Kernel Thread Stacks



Each OS161 thread has two stacks, one that is used while the thread is executing unprivileged application code, and another that is used while the thread is executing privileged kernel code.

# OS161 Trap Frame



While the kernel handles the system call, the application's CPU state is saved in a trap frame on the thread's kernel stack, and the CPU registers are available to hold kernel execution state.

# The OS161 Thread Interface (incomplete)

- `/* Make a new thread, which will start executing at "func". The thread will belong to the process "proc", or to the current thread's process if "proc" is null. The "data" arguments (one pointer, one *number) are passed to the function. */`  
`int thread_fork (const char *name, struct proc *proc,`  
`void (*entrypoint)(void *, unsigned long),`  
`void *data1, unsigned long data2);`
- `/* Cause the current thread to exit. Interrupts need not be disabled. */`  
`__DEAD void thread_exit(void);`
- `/* Cause the current thread to yield to the next runnable thread, but itself stay runnable. Interrupts need not be disabled. */`  
`void thread_yield(void);`

# Creating Threads Using `thread_fork()`

```
runthreads(int doloud){
    char name[16];
    int i, result;
    for (i=0; i<NTHREADS; i++) {
        snprintf(name, sizeof(name), "threadtest%d", i);
        result = thread_fork(name, NULL,
                             doloud ? loudthread : quietthread, NULL, i);
        if (result) {
            panic("threadtest: thread_fork failed %s)\n",
                  strerror(result));
        }
    }
    for (i=0; i<NTHREADS; i++) {
        P(tsem);
    }
}
```



# From thread fork() to thread execution (ready state)

```
thread_fork(..., void (*entrypoint)(void *, unsigned long),
            void *data1, unsigned long data2) {
    ...
    newthread = thread_create(...);
    ...
    switchframe_init(newthread, entrypoint, data1, data2);
    thread_make_runnable(newthread, false);
}
thread_create(...) {
    thread = kmalloc(sizeof(*thread));
    thread->... = ...;
    return thread;
}
switchframe_init(...) {
    /* setup switchframe in stack */
}
thread_make_runnable(struct thread *target) {
    ...
    target->t_state = S_READY;
    threadlist_addtail(&targetcpu->c_runqueue, target);
    ...
}
```

# From ready to execution: `thread_switch()`

```
thread_switch(threadstate_t newstate, ...) {
    struct thread *cur, *next;

    cur = curthread;
    /* Put the thread in the right place. */
    switch (newstate) {
        case S_RUN:
            panic("Illegal S_RUN in thread_switch\n");
        case S_READY:
            thread_make_runnable(cur, true /*have lock*/);
            break;
    }
    next = threadlist_remhead(&curcpu->c_runqueue);

    /* do the switch (in assembler in switch.S) */
    switchframe_switch(&cur->t_context, &next->t_context);
    ...
}
```

# Review: MIPS Register Usage

See also: `kern/arch/mips/include/kern/regdefs.h`

R0, zero = ## zero (always returns 0)

R1, at = ## reserved for use by assembler

R2, v0 = ## return value / system call number

R3, v1 = ## return value

R4, a0 = ## 1st argument (to subroutine)

R5, a1 = ## 2nd argument

R6, a2 = ## 3rd argument

R7, a3 = ## 4th argument

# Review: MIPS Register Usage

```
R08-R15, t0-t7 = ## temps (not preserved by subroutines)
R24-R25, t8-t9 = ## temps (not preserved by subroutines)
                  ## can be used without saving
R16-R23, s0-s7 = ## preserved by subroutines
                  ## save before using,
                  ## restore before return
R26-27,  k0-k1 = ## reserved for interrupt handler
R28,     gp =    ## global pointer
                  ## (for easy access to some variables)
R29,     sp =    ## stack pointer
R30,     s8/fp   ## 9th subroutine reg / frame pointer
R31,     ra =    ## return addr (used by jal)
```

# Dispatching on the MIPS (1 of 2)

```
/* see kern/arch/mips/thread/switch.S */
switchframe_switch:
    /* a0/a1 point to old/new thread's switchframe (control block) */

    /* Allocate stack space for saving 10 registers. 10*4 = 40 */
    addi sp, sp, -44

    /* Save the registers */
    sw ra, 36(sp)
    sw gp, 32(sp)
    sw s8, 28(sp)
    sw s6, 24(sp)
    sw s5, 20(sp)
    sw s4, 16(sp)
    sw s3, 12(sp)
    sw s2, 8(sp)
    sw s1, 4(sp)
    sw s0, 0(sp)
    /* Store the old stack pointer in the old thread */
    sw sp, 0(a0)
```

# Dispatching on the MIPS (2 of 2)

```
/* Get the new stack pointer from the new thread */
lw sp, 0(a1)
nop /* delay slot for load */

/* Now, restore the registers */
lw s0, 0(sp)
lw s1, 4(sp)
lw s2, 8(sp)
lw s3, 12(sp)
lw s4, 16(sp)
lw s5, 20(sp)
lw s6, 24(sp)
lw s8, 28(sp)
lw gp, 32(sp)
lw ra, 36(sp)
nop /* delay slot for load */

j ra /* and return. */
addi sp, sp, 40 /* in delay slot */
.end switchframe_switch
```

# mips trap: Handling System Calls, Exceptions, and Interrupts

- On the MIPS, the same exception handler is invoked to handle system calls, exceptions and interrupt
- The hardware sets a code to indicate the reason (system call, exception, or interrupt) that the exception handler has been invoked
- OS161 has a handler function corresponding to each of these reasons. The mips trap function tests the reason code and calls the appropriate function: the system call handler (mips syscall) in the case of a system call.
- Mips trap can be found in kern/arch/mips/mips/trap.c.

# OS161 MIPS System Call Handler

```
mips_syscall(struct trapframe *tf) {
    assert(curspl==0);
    callno = tf->tf_v0; retval = 0;
    switch (callno) {
        case SYS_reboot:
            err = sys_reboot(tf->tf_a0); /* in kern/main/main.c */
            break;

        /* Add stuff here */
        default:
            kprintf("Unknown syscall %d\n", callno);
            err = ENOSYS;
            break;
    }
}
```

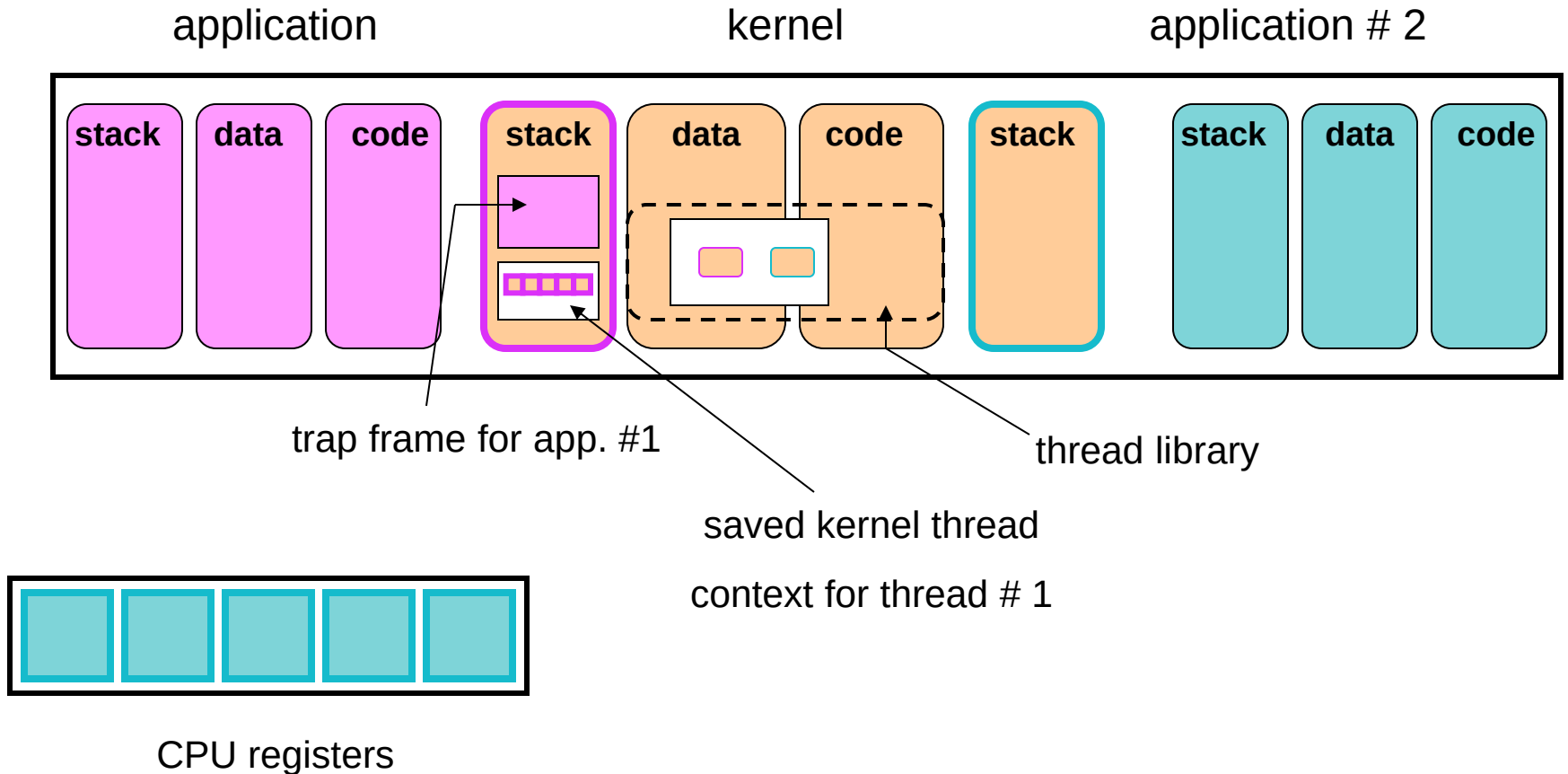


# MIPS Exceptions

|         |    |                                            |
|---------|----|--------------------------------------------|
| EX_IRQ  | 0  | /* Interrupt */                            |
| EX_MOD  | 1  | /* TLB Modify (write to read-only page) */ |
| EX_TLBL | 2  | /* TLB miss on load */                     |
| EX_TLBS | 3  | /* TLB miss on store */                    |
| EX_ADEL | 4  | /* Address error on load */                |
| EX_ADES | 5  | /* Address error on store */               |
| EX_IBE  | 6  | /* Bus error on instruction fetch */       |
| EX_DBE  | 7  | /* Bus error on data load *or* store */    |
| EX_SYS  | 8  | /* Syscall */                              |
| EX_BP   | 9  | /* Breakpoint */                           |
| EX_RI   | 10 | /* Reserved (illegal) instruction */       |
| EX_CPU  | 11 | /* Coprocessor unusable */                 |
| EX_OVF  | 12 | /* Arithmetic overflow */                  |

In OS161, mips trap uses these codes to decide whether it has been invoked because of an interrupt, a system call, or an exception.

# Two Processes in OS161



# System Calls for Process Management

|                 | linux                            | OS161      |
|-----------------|----------------------------------|------------|
| Creation        | fork,execve                      | fork,execv |
| Destruction     | _exit,kill                       | _exit      |
| Synchronization | wait,waitpid,pause,...           | waitpid    |
| Attribute Mgmt  | getpid,getuid,nice,getrusage,... | getpid     |

# The MIPS R3000 TLB

- The MIPS has a software-controlled TLB than can hold 64 entries.
- Each TLB entry includes a virtual page number, a physical frame number, an address space identifier (not used by OS161), and several flags (valid, read-only)
- OS161 provides low-level functions for managing the TLB:

**TLB Write:** modify a specified TLB entry

**TLB Random:** modify a random TLB entry

**TLB Read:** read a specified TLB entry

**TLB Probe:** look for a page number in the TLB

- If the MMU cannot translate a virtual address using the TLB it raises an exception, which must be handled by OS161

See `kern/arch/mips/include/tlb.h`

# OS161 Address Spaces: dumbvm

- OS161 starts with a very simple virtual memory implementation
- virtual address spaces are described by `addrspace` objects, which record the mappings from virtual to physical addresses

```
struct addrspace {  
#if OPT_DUMBVM  
    vaddr_t as_vbase1; /* base virtual address of code segment */  
    paddr_t as_pbase1; /* base physical address of code segment */  
    size_t as_npages1; /* size (in pages) of code segment */  
    vaddr_t as_vbase2; /* base virtual address of data segment */  
    paddr_t as_pbase2; /* base physical address of data segment */  
    size_t as_npages2; /* size (in pages) of data segment */  
    paddr_t as_stackbase; /* base physical address of stack */  
#else  
    /* Put stuff here for your VM system */  
#endif  
};
```

This amounts to a slightly generalized version of simple dynamic relocation, with three bases rather than one.

# Address Translation Under dumbvm

- the MIPS MMU tries to translate each virtual address using the entries in the TLB
- If there is no valid entry for the page the MMU is trying to translate, the MMU generates a page fault (called an *address exception*)
- The vm fault function (see kern/arch/mips/mips/dumbvm.c) handles this exception for the OS161 kernel. It uses information from the current process' address space to construct and load a TLB entry for the page.
- On return from exception, the MIPS retries the instruction that caused the page fault. This time, it may succeed.

vm fault is not very sophisticated. If the TLB fills up, OS161 will crash!

# An Address Space for the Kernel

- Each process has its own address space. What about the kernel?
- two possibilities
  - **Kernel in physical space: disable address translation in privileged system**
    - execution mode, enable it in unprivileged mode
  - **Kernel in separate virtual address space: need a way to change address**
    - translation (e.g., switch page tables) when moving between privileged and unprivileged code
- OS161, Linux, and other operating systems use a third approach: the kernel is mapped into a portion of the virtual address space of *every process*
- memory protection mechanism is used to isolate the kernel from applications
- one advantage of this approach: application virtual addresses (e.g., system call parameters) are easy for the kernel to use

# Loading a Program into an Address Space

- When the kernel creates a process to run a particular program, it must create an address space for the process, and load the program's code and data into that address space
- A program's code and data is described in an *executable file*, which is created when the program is compiled and linked
- OS161 (and other operating systems) expect executable files to be in ELF(**E**xecutable and **L**inking **F**ormat)format
- the OS161 `execv` system call, which re-initializes the address space of a process  

```
#include <unistd.h>
int execv(const char *program, char **args)
```
- The `program` parameter of the `execv` system call should be the name of the ELF executable file for the program that is to be loaded into the address space.



# ELF Files

- ELF files contain address space segment descriptions, which are useful to the kernel when it is loading a new address space
- the ELF file identifies the (virtual) address of the program's first instruction
- the ELF file also contains lots of other information (e.g., section descriptors, symbol tables) that is useful to compilers, linkers, debuggers, loaders and other tools used to build programs

# Address Space Segments in ELF Files

- Each ELF segment describes a contiguous region of the virtual address space.
- For each segment, the ELF file includes a segment *image* and a header, which describes:
  - the virtual address of the start of the segment
  - the length of the segment in the virtual address space
  - the location of the start of the image in the ELF file
  - the length of the image in the ELF file
- the image is an exact copy of the binary data that should be loaded into the specified portion of the virtual address space
- the image may be smaller than the address space segment, in which case the rest of the address space segment is expected to be zero-filled

To initialize an address space, the kernel copies images from the ELF file to the specified portions of the virtual address space

# ELF Files and OS161

- OS161's dumbvm implementation assumes that an ELF file contains two segments:
  - a *text segment*, containing the program code and any read-only data
  - a *data segment*, containing any other global program data
- the ELF file does not describe the stack (why not?)
- dumbvm creates a *stack segment* for each process. It is 12 pages long, ending at virtual address 0x7fffffff

Look at kern/userprog/loadelf.c to see how OS161 loads segments from ELF files

# ELF Sections and Segments

- In the ELF file, a program's code and data are grouped together into *sections*, based on their properties. Some sections:
  - **.text**: program code
  - **.rodata**: read-only global data
  - **.data**: initialized global data
  - **.bss**: uninitialized global data (Block Started by Symbol)
  - **.sbss**: small uninitialized global data
- not all of these sections are present in every ELF file
- normally
  - the .text and .rodata sections together form the text segment
  - the .data, .bss and .sbss sections together form the data segment
- space for *local* program variables is allocated on the stack when the program runs

# The segments.c Example Program (1 of 2)

```
#include <unistd.h>

#define N    (200)

int x = 0xdeadbeef;
int y1;
int y2;
int y3;
int array[4096];
char const *str = "Hello World\n";
const int z = 0xabcdcdcba;

struct example {
    int ypos;
    int xpos;
};
```

# The segments.c Example Program (2 of 2)

```
int
main()
{
    int count = 0;
    const int value = 1;
    y1 = N;
    y2 = 2;
    count = x + y1;
    y2 = z + y2 + value;

    reboot(RB_POWEROFF);
    return 0; /* avoid compiler warnings */
}
```

# ELF Sections for the Example Program

Section Headers:

| [Nr] | Name     | Type         | Addr     | Off    | Size   | ES | Flg |
|------|----------|--------------|----------|--------|--------|----|-----|
| [ 0] |          | NULL         | 00000000 | 000000 | 000000 | 00 |     |
| [ 1] | .reginfo | MIPS_REGINFO | 00400094 | 000094 | 000018 | 18 | A   |
| [ 2] | .text    | PROGBITS     | 004000b0 | 0000b0 | 000200 | 00 | AX  |
| [ 3] | .rodata  | PROGBITS     | 004002b0 | 0002b0 | 000020 | 00 | A   |
| [ 4] | .data    | PROGBITS     | 10000000 | 001000 | 000010 | 00 | WA  |
| [ 5] | .sbss    | NOBITS       | 10000010 | 001010 | 000014 | 00 | WAp |
| [ 6] | .bss     | NOBITS       | 10000030 | 00101c | 004000 | 00 | WA  |
| [ 7] | .comment | PROGBITS     | 00000000 | 00101c | 000036 | 00 |     |

...

Flags: W (write), A (alloc), X (execute), p (processor specific)

## Size = number of bytes (e.g., .text is 0x200 = 512 bytes)

## Off = offset into the ELF file

## Addr = virtual address

The mips-harvard-os161-readelf program can be used to inspect OS161 MIPS ELF files: mips-harvard-os161-readelf -a segments

# ELF Segments for the Example Program

Program Headers:

| Type    | Offset   | VirtAddr   | PhysAddr   | FileSiz | MemSiz  | Flg | Align  |
|---------|----------|------------|------------|---------|---------|-----|--------|
| REGINFO | 0x000094 | 0x00400094 | 0x00400094 | 0x00018 | 0x00018 | R   | 0x4    |
| LOAD    | 0x000000 | 0x00400000 | 0x00400000 | 0x002d0 | 0x002d0 | R E | 0x1000 |
| LOAD    | 0x001000 | 0x10000000 | 0x10000000 | 0x00010 | 0x04030 | RW  | 0x1000 |

- segment info, like section info, can be inspected using the `cs350-readelf` program
- the REGINFO section is not used
- the first LOAD segment includes the `.text` and `.rodata` sections
- the second LOAD segment includes `.data`, `.sbss`, and `.bss`



# Contents of the Example Program's .text Section

Contents of section .text:

```
4000b0 3c1c1001 279c8000 3c08ffff 3508fff8 <...'.<...5...  
...  
## Decoding 3c1c1001 to determine instruction  
## 0x3c1c1001 = binary 1111000001110000010000000000001  
## 0011 1100 0001 1100 0001 0000 0000 0001  
## instr  | rs      | rt      | immediate  
## 6 bits | 5 bits| 5 bits| 16 bits  
## 001111 | 00000 | 11100 | 0001 0000 0000 0001  
## LUI     | 0      | reg 28| 0x1001  
## LUI     | unused| reg 28| 0x1001  
## Load unsigned immediate into rt (register target)  
## lui gp, 0x1001
```

The mips-harvard-os161-objdump program can be used to  
inspect OS161 MIPSELF file section contents:  
mips-harvard-os161-objdump -s segments

# Contents of the Example Program's .rodata Section

Contents of section .rodata:

```
4002b0 48656c6c 6f20576f 726c640a 00000000 Hello World.....
4002c0 abcddcba 00000000 00000000 00000000 .....
...
## 0x48 = 'H' 0x65 = 'e' 0x0a = '\n' 0x00 = '\0'
## Align next int to 4 byte boundary
## const int z = 0xabcddcba
## If compiler doesn't prevent z from being written,
## then the hardware could
## Size = 0x20 = 32 bytes "Hello World\n\0" = 13 + 3 padding = 16
## + const int z = 4 = 20
## Then align to the next 16 byte boundry at 32 bytes.
```

The .rodata section contains the "Hello World" string literal and the constant integer variable z.

# Contents of the Example Program's .data Section

Contents of section .data:

```
10000000 deadbeef 004002b0 00000000 00000000 .....@.....
```

```
...
```

```
## Size = 0x10 bytes = 16 bytes
```

```
## int x = deadbeef (4 bytes)
```

```
## char const *str = "Hello World\n"; (4 bytes)
```

```
## value stored in str = 0x004002b0.
```

```
## NOTE: this is the address of the start
```

```
## of the string literal in the .rodata section
```

The .data section contains the initialized global variables str  
and x.

# Contents of the Example Program's .bss and .sbss Sections

```
...
10000010 A __bss_start
10000010 A _edata
10000010 A _fbss
10000010 S y3    ## S indicates sbss section
10000014 S y2
10000018 S y1
1000001c S errno
10000020 S __argv
...
10000030 B array  ## B indicates bss section
10004030 A _end
```

The y1, y2, and y3 variables are in the .sbss section. The array variable is in the .bss section. There are no values for these variables in the ELF file, as they are uninitialized. The mips-harvard-os161-nm program can be used to inspect symbols defined in ELF files: mips-harvard-os161-nm -b segments